Ask Devin about Ip-59/TeachAI

Fast

Menu

Entrypoints: run_teachai.py and TeachAI.ipynb

Relevant source files

Purpose and Scope

This page details the entrypoint mechanisms for the TeachAI system, encompassing both the interactive Jupyter Notebook (**`TeachAI.ipynb`**) and the command-line interface (**`run_teachai.py`**). It covers the architectural distinction between **`start_jupyter()`** and **`main()`** , the dependency-aware module reload sequence designed for iterative Jupyter development, and the initialization lifecycle managed by **`TeachAIEngine`** .

Sources: **`[TeachAI.ipynb:1-50]`** , **`[run_teachai.py:1-121]`** , `engine.py` | `23-47`

Jupyter Notebook Entrypoint (`TeachAI.ipynb`)

The primary user-facing entrypoint for the inference system is the Jupyter notebook **`TeachAI.ipynb`** . It initializes the environment, loads dependencies, and invokes the core startup routine without requiring command-line access.

Notebook Cell Structure

The notebook consists of an introductory markdown cell documenting usage prerequisites (such as setting up **`.env`** and installing **`requirements.txt`**) followed by a single execution cell that imports and executes **`start_jupyter()`** :

```
from run_teachai import start_jupyter

start_jupyter()
```

Sources: `TeachAI.ipynb` | `1-34`

Ask Devin about Ip-59/TeachAI

Fast

The module `run_teachai.py` acts as the single entrypoint multiplexer, separating execution paths for Jupyter (`start_jupyter()`) and command-line execution (`main()`).

Sources: `[run_teachai.py:1-121]` , engine.py | 65-164

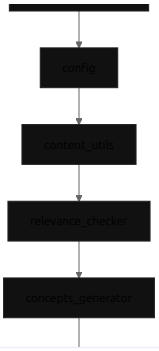
Execution Functions

- `start_jupyter(*, reload_modules: bool = False)` : Designed to run within a Jupyter kernel session. It invokes `load_dotenv(override=True)` `[run_teachai.py:64]` , applies OpenAI connection environments `[run_teachai.py:68]` , optionally triggers `reload_project_modules()` `[run_teachai.py:74]` , and instantiates `TeachAIEngine` to return the root interactive widget (`VBox`) `[run_teachai.py:76-84]` .
- `main()` : Designed for CLI execution (`python run_teachai.py`). It loads environment variables, instantiates `TeachAIEngine` , executes `engine.initialize()` to validate configuration and API keys, and boots up the system components [run_teachai.py:90-117].

Sources: `[run_teachai.py:53-120]` , engine.py | 65-164

Module Reload Order for Jupyter Development (`_RELOAD_ORDER`)

During active Jupyter development, updating backend `.py` files without restarting the kernel requires reloading modules into `sys.modules` in a strict topological order. `run_teachai.py` defines `_RELOAD_ORDER` to prevent broken references during hot-reloads.



Ask Devin about Ip-59/TeachAI

Fast

Reload Mechanics

The `reload_project_modules()` function checks `sys.modules` for each entry in `_RELOAD_ORDER` and applies `importlib.reload(module)` [run_teachai.py:36-51].

- **Topological Sequence**: Fundamental utilities (`config` , `content_utils`) load first, followed by generators (`content_generator` , `lesson_generator`), and finally orchestration and UI layers (`state_manager` , `engine` , `interface`) [run_teachai.py:20-33].
- **Safety Default**: `reload_modules` defaults to `False` in `start_jupyter()` because hot-reloading active `ipywidgets` can occasionally cause UI deadlocks [run_teachai.py:53-60].

Sources: run_teachai.py | 10-51

Ask Devin about Ip-59/TeachAI

Fast

Ask Devin about Ip-59/TeachAI

Fast